DEBUGGING, BREAKPOINTS AND WATCHPOINTS

A debugger is a software tailored to analyze in detail the behavior of the code, with the specific goal to support the fixing of programming bugs. The main task of the debugger is to **show a fragment of the code that is generating the problems** (at source level or down at the assembly level), by exploiting a disassembler possibly integrated in the debugger toolflow. To analyze the code, it is in general necessary a **specific compilation mode of the application program**, where more instructions are introduced to support the execution in **debug-mode**, that is typically much slower than the execution at regular speed on the target processor. The majority of the debuggers support a **step-by-step** execution of the code, showing the instruction currently executed, the status of the CPU or the value of some selected variables, or part of the memory locations. The execution can be interrupted on a per instruction base, or at specific locations specified by the user **-breakpoints or watchpoints**- where the status of the machine need to be analysed more in detail. Typically, the instructions before a crash are those more important, but for other types of analysis any possible excerpt of code can be of interest. More sophisticated debuggers provide the possibility to modify the execution flow of the code by exploiting ad-hoc hardware support implemented in the CPU, for the analysis and control of the program execution. Breakpoints and watchpoints enable the programmer to **stop the target when certain events occur, and when certain conditions are met**. When execution stops it is possible to examine the contents of memory, registers, or variables, or to take some specified actions before execution resumes.

A ***Hardware Breakpoint (HWBP)*** is in practice implemented through some special logic that is integrated into the device. A hardware breakpoint can be viewed as a **set of programmable comparators** that are connected to the **program address bus**. These comparators are programmed with a specific address value. When the code is executing, and all of the bits in the address on the program address bus match the bits programmed into the comparators, the **Hardware breakpoint logic generates a signal to the CPU to Halt**. The advantage of using a hardware breakpoint is that it can be used for **any type of memory**, not only using volatile memory as in the case of the *software breakpoints*. Hardware Breakpoints can be used regardless of whether the code being executed is in RAM or ROM, because to the hardware breakpoint logic, there is no difference. It is just matching an address on the Program Address Bus and halting the CPU when it finds one. The disadvantage of the HWBP is the **limited number available**, because they are implemented in hardware. The number of HWBPs available differs from architecture to architecture, but in most cases there are only 2-8 available.

A ***Software Breakpoint (SWBP)*** is implemented in software. There are actually a 2 main different implementations. Some devices reserve **a specified bit** in their opcode definition that indicates a Software breakpoint. As an example, in one architecture of the Texas Instrument C6000 family, all instructions are 32 bits long, and bit 28 is reserved to indicate a software breakpoint, so that all instructions in that instruction set have bit 28 as a zero. In this case, when a software **breakpoint is set**, it will actually **modify the opcode of the instruction** at that location and set bit 28 to a 1. The Emulation logic then monitors the Program Opcode for whenever bit 28 is a 1, and halts the CPU when that occurs. Note that this is a minority case. Most architectures don't do it this way. The reason is that it limits the flexibility of the instruction set. Also, it doesn't work for architectures that have **variable length instructions**, **so it also limits code density**. The more popular way of implementing a software breakpoint is also much more complex. In this scenario, there is a specified **breakpoint opcode**. Typically, the opcode is 8-bits. Whenever a breakpoint is set, the leading 8 bits of the instruction at that location are removed and **replaced with this 8-bit breakpoint opcode**. The original 8-bits of the instruction are then stored in a breakpoint table. Whenever a breakpoint is encountered, the **CPU halts and the debugging tool replaces the breakpoint opcode with the original 8-bits of the instruction**. When execution is restarted, the tool must do a bit of trickery because the instruction in the actual CPU pipeline isn't correct. It still has the breakpoint opcode. So the tool flushes the CPU pipeline and then re-fetches the instructions that were pending in them to their original state, with the next function to be executed being the one where the breakpoint was set. At the same time, the tool re-loads the instruction at that location with the breakpoint opcode so that the next time this code is encountered, it will again halt. The advantage of the SWBP is that there is an **unlimited number of them**, so it is possible to put them in any place. The disadvantage is that it is **not possible to put them in non-volatile memory** like ROM/FLASH, etc, because the debugging tool normally cannot write the opcode to the location.

As a commercial example, let us consider the support available in the **ARM DS-5** debugger.

Breakpoints A breakpoint enables to interrupt an application when execution reaches a specific address. A breakpoint is always related to a particular memory address, regardless of what might be stored there. When execution reaches the breakpoint, normal execution stops before any instruction stored there is performed. It is possible to set:

- software breakpoints that trigger when a particular instruction is executed at a specific address

- hardware breakpoints that trigger when the processor attempts to execute an instruction that is fetched from a specific memory address
- conditional breakpoints that trigger when an expression evaluates to true or when an ignore counter is reached
- temporary software or hardware breakpoints that are subsequently deleted when the breakpoint is hit.

The type of breakpoints which can be set depends on the:

- memory region and the related access attributes
- hardware support provided by the target processor
- debug interface used to maintain the target connection
- running state if the programmers is debugging an OS aware application.

Watchpoints A watchpoint is similar to a breakpoint, but **it is the address or value of a data access that is monitored** rather than an instruction being executed from a specific address. It is possible to specify a register or a memory address to identify a location that is to have its contents tested. Watchpoints are sometimes known as **data breakpoints**, emphasizing that they are data dependent. Execution of the application stops when the address being monitored is accessed. It is possible to set:

- watchpoints that trigger when a particular memory location is accessed in a particular way
- conditional watchpoints that trigger when an expression evaluates to true or when an ignore counter is reached.

The **hardware of the microcontrollers** typically contains specific modules simplifying the implementation of breakpoints and watch-points.